WIDTH OPENS A MEMORISATION GATE AND SETS A WEIGHT-DECAY CLOCK: AN EMPIRICAL ACCOUNT OF MODEL SCALE IN GROKKING

Anonymous authors
Paper under double-blind review

ABSTRACT

Grokking—validation accuracy rising long after the training set is fit—is usually explained through circuit efficiency, lazy-to-rich dynamics or weight norm, but its dependence on *model size* is rarely characterised systematically. We study modular addition with one-layer transformers whose width spans 16× (1.8k–213k parameters) and test a pre-registered, falsifiable account: (i) a *capacity gate*: only models large enough to store the training labels "memorise first"; (ii) above the gate, wider models generalise sooner, but the gain saturates; (iii) the floor is set by an *optimiser clock* $\propto 1/(\mathrm{lr} \cdot \mathrm{wd})$, not by the model. With 8 seeds per width: memorise-first behaviour appears only above $\approx 10^4$ parameters (for 1,392 training pairs); just above it the delay is longest and 5/8 runs at d=24 do not generalise in 30k steps; then the median time to generalise falls as $P^{-0.46}$ (95% CI $[-0.66, -0.26]$) for $d \geq 32$ and is statistically flat for $d \geq 64$ (-0.21, $[-0.56, 0.04]$), while the delay relative to the fitting time does *not* grow. The time to generalise scales as $(\mathrm{lr} \cdot \mathrm{wd})^{-1.2}$, so per-width tuning removes most of the width effect among larger models, and the total parameter norm at generalisation is nearly width-independent (CV 5.6%). Parts of the account fail: the gate size is not a function of the number of training pairs alone (it scales as $N^{1.04}$ over four low-data configurations but is absent up to $3{\times}10^4$ parameters when 60–70% of pairs are seen), extra depth does not speed up generalisation, and a rescaling intervention supports a causal role of norm only one-sidedly. Compute was 4 CPU cores, so seed counts are small; we flag every conclusion that is within noise.

1 INTRODUCTION

Power et al. (Power et al., 2022) observed that small networks trained on algorithmic tables such as modular addition can fit the training set within a few hundred steps and then, after orders of magnitude more training, abruptly generalise. Explanations include a competition between a fast-to-learn but parameter-inefficient memorising circuit and a slower, more efficient generalising one (Varma et al., 2023; Nanda et al., 2023), a transition from lazy to rich training dynamics (Kumar et al., 2024), and a mismatch between the weight-norm dependence of training and test loss (Liu et al., 2023). Most of these accounts hold the architecture fixed and vary data fraction, weight decay or initialisation scale. What the *size* of the model does to the delay is less systematically characterised—even though practitioners choose a size before anything else, and size is the control variable on which memorisation capacity (Zhang et al., 2017; Morris et al., 2025) and double descent (Nakkiran et al., 2020; Davies et al., 2023) depend.

This paper. We ask: *on modular arithmetic, how does the appearance and the length of the grokking delay depend on model size?* Rather than reporting a trend, we commit in advance (Appendix A) to a mechanistic account, derive predictions that can fail, and test them on fresh seeds and settings. The account:

H-CAP **Capacity gate.** Memorising N labels of $\log_2 p$ bits needs roughly $N \log_2 p$ bits of parameters. Below a critical size $P^*(N)$ a model cannot memorise first: it either fails to fit or fits

while already generalising, so there is no delay to speak of. Above P^* it can, and the delay appears. Predicts $P^* \propto N$.

H-SPEED **Bigger is faster above the gate, with saturation.** At fixed optimiser settings the time to generalise decreases with width, but the gain is limited by the optimiser's timescale, $t_{\text{gen}} \propto (\text{lr} \cdot \text{wd})^{-1}$.

H-NORM **Norm threshold.** (Formed after inspecting weight norms in the first sweep.) Generalisation sets in when the *total* parameter norm decays to a data-dependent value that does not depend on width.

Contributions. (1) A width sweep with 8 seeds per size, reporting censoring-aware medians and bootstrap CIs, shows three regimes: no memorise-first at small sizes, a high-variance transition with large delays, and a faster, saturating regime. (2) The time to generalise is controlled by $(\text{lr} \cdot \text{wd})^{\approx -1.2}$ and width only enters through a prefactor that saturates, so *conclusions about scale depend on whether optimiser settings are held fixed*. (3) Weight norm at generalisation is nearly width-independent, but our intervention experiment gives only one-sided causal support. (4) A scorecard of which pre-registered predictions held, failed or could not be tested (Table 3). Everything was produced on 4 CPU cores with small models; the limits this implies are discussed in Section 6.

2 Related work

Grokking. Grokking was reported on modular and permutation tasks by Power et al. (2022). Nanda et al. (2023) reverse-engineer a one-layer transformer on modular addition into a discrete-Fourier-transform circuit and define progress measures; we reuse their architecture and optimiser settings. Liu et al. (2022) build an effective theory of representation learning and a phase diagram over hyperparameters; Liu et al. (2023) link grokking to a "LU" mismatch between train and test loss as a function of weight norm. Varma et al. (2023) explain grokking by circuit efficiency (generalising circuits produce larger logits for a given norm) and test predictions on data size, including "ungrokking". Kumar et al. (2024) tie it to the lazy-to-rich transition. Merrill et al. (2023) study sparse versus dense subnetworks on sparse parities, Thilak et al. (2022) the role of adaptive optimisers, and Zhong et al. (2023) show that small changes to hyper-parameters or initialisation yield different algorithms on modular addition. These works mostly vary data fraction, weight decay or initialisation; we instead vary model size and read the results through their optimiser and norm accounts.

Capacity, memorisation and scale. Networks can memorise random labels (Zhang et al., 2017); the information a model can store scales with its parameter count (Morris et al., 2025); and larger language models memorise training data faster and can memorise more before overfitting (Tirumala et al., 2022). Nakkiran et al. (2020) and Davies et al. (2023) relate test error to model size and training time; the latter unify double descent with grokking. We use the capacity viewpoint to derive a size gate for memorise-first behaviour, and test it directly rather than assuming it.

Optimisation. Weight decay in AdamW (Loshchilov & Hutter, 2019) shrinks parameters at a rate $\text{lr} \cdot \text{wd}$ per step; this sets the clock we find to dominate the large-width regime.

3 Setup

Task and data. Inputs are all p^2 pairs (a, b) with $a, b \in \{0, \ldots, p-1\}$ and label $(a + b) \bmod p$; data are generated in memory, no external data are used. Unless stated, p=59 and a uniformly random 40% of pairs (N=1,392) forms the training set; the remaining 2,089 are validation. The split is re-drawn for every seed (together with the initialisation).

Models. Main: a one-layer, decoder-only transformer in the style of Nanda et al. (2023) (Vaswani et al., 2017), without LayerNorm, 4 heads of dimension $d/4$, ReLU MLP of width $4d$, learned token and position embeddings, input $[a, b, =]$ and readout at the last position. Weights are initialised

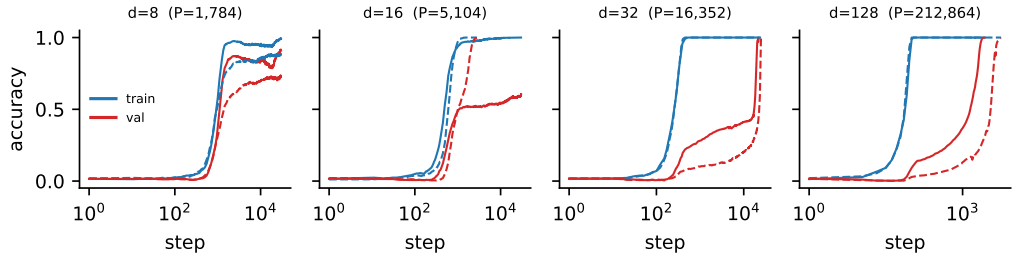


Figure 1: Train (blue) and validation (red) accuracy for two seeds (solid/dashed) at four widths (p=59, N=1,392, lr 10^{-3}, wd 1). Smallest model: partial generalisation during fitting; d=16: fit and partial generalisation coincide, a plateau at ≈ 0.5 for some seeds; larger models: classic memorise-then-generalise.

$\mathcal{N}(0, 1/\text{fan-in})$. We vary $d_{\text{model}} \in \{8, 16, 24, 32, 48, 64, 96, 128\}$, i.e. $P \in [1.8\text{k}, 213\text{k}]$ parameters. In Section 4.5 we also use a 2-hidden-layer ReLU MLP on concatenated embeddings, deeper transformers, and decouple d_{mlp} from d_{model}.

Training. Full-batch AdamW (Loshchilov & Hutter, 2019) (β=$(0.9, 0.98)$, lr 10^{-3} with 10-step warm-up, weight decay 1.0), cross-entropy, float32, up to 30,000 steps; training stops 300 steps after both accuracies are ≥ 0.99. Accuracies, losses, parameter L2 norm and the Fourier concentration of the token embedding (fraction of mean-centred embedding energy in its 5 strongest frequencies) are logged every 50 steps (every 10 before step 400).

Measures. t_{fit} (t_{gen}) is the first logged step from which training (validation) accuracy stays ≥ 0.99 for three consecutive logs (with a 3% tolerance on the later two). $a_{\text{fit}}^{\text{val}}$ is validation accuracy at t_{fit}. A run is *memorise-first* if it fits the training set and $a_{\text{fit}}^{\text{val}} < 0.5$. The delay is $D = t_{\text{gen}} - t_{\text{fit}}$. Runs that do not reach t_{gen} in the budget are *censored*; medians treat them as $+\infty$ and are reported as ">30k" when the median itself is censored. Slopes are least-squares fits of $\log t$ against $\log P$ to per-width medians, with 95% bootstrap intervals obtained by resampling seeds within each width (percentile method, 2,000 resamples; resamples with an infinite median are discarded and counted).

Compute and statistics. All experiments ran on 4 CPU cores (cgroup quota) in single-threaded processes; a full run takes 1–25 minutes. Seeds: 8 per width in the main sweep, 3–4 elsewhere (stated per experiment). With so few seeds, intervals are wide and we avoid claims about differences that they do not resolve.

4 Results

4.1 Width sweep: three regimes

Figure 1 shows representative trajectories and Figure 2 and Table 1 summarise 8 seeds per width.

No memorise-first below the gate. At d=8 (P=1.8k) no run is memorise-first: half the runs never fit, and the ones that fit have $a_{\text{fit}}^{\text{val}} \approx 0.8$–$0.9$. At d=16 (5.1k) $a_{\text{fit}}^{\text{val}} \approx 0.5$; 3/8 runs are memorise-first, and 4/8 runs are still on a partial-generalisation plateau at 30k steps (val 0.47–0.60). From d=24 (10k) all 8 runs memorise first, and $a_{\text{fit}}^{\text{val}}$ falls smoothly from 0.23 (d=24) to ≈ 0.05 ($d \geq 64$). The Fourier concentration of the embedding at t_{fit} falls from 0.9 to 0.24 with width, i.e. large models fit with an unstructured embedding while small ones already use Fourier features (Fig. 2f).

The delay is non-monotone in size. Just above the gate the delay is the longest and the most variable: at d=24 only 3/8 runs generalise in 30k steps, at d=32 and 48 7/8 do, with median t_{gen} of 19,400 (CI [10,000, 24,700]) and 8,900 ([5,700, 23,200]). From d=64 all runs generalise, $t_{\text{gen}} \approx$ 6,000 and falls slowly to 5,025 ([3,300, 6,200]) at d=128. For $d \geq 32$ the slope of $\log t_{\text{gen}}$ against

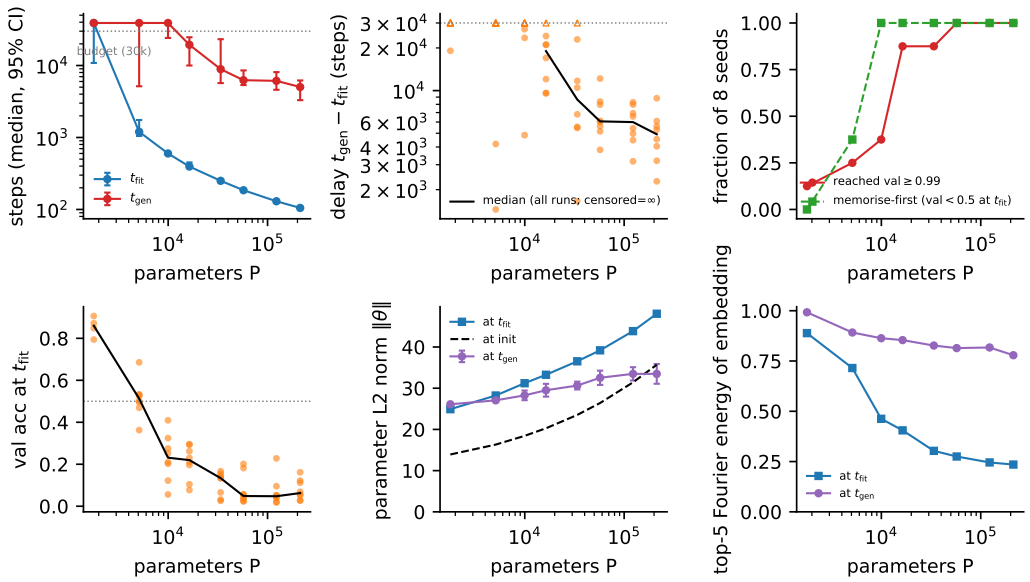


Figure 2: Width sweep, 8 seeds per width. (a) Median t_{fit} and t_{gen} with 95% bootstrap CIs (censored medians drawn at the cap). (b) Per-run delay $t_{\text{gen}} - t_{\text{fit}}$; triangles are runs that never generalised. (c) Fraction of seeds that reach val≥0.99 and that are memorise-first. (d) Validation accuracy at t_{fit} per run. (e) Parameter norm at initialisation, at t_{fit} and at t_{gen}. (f) Fraction of token-embedding energy in the 5 strongest Fourier modes.

Table 1: Width sweep (Fig. 2); medians over 8 seeds. "fit/gen": runs reaching train≥0.99 / val≥0.99 within 30k steps. t_{gen} in brackets: 95% bootstrap CI of the median. "mem.": memorise-first runs. $\|\theta\|_{\text{gen}}$: mean parameter norm at t_{gen} over generalising runs.

d	P	fit/gen	t_{fit}	t_{gen} [95% CI]	$a_{\text{fit}}^{\text{val}}$	mem.	$\|\theta\|_{\text{gen}}$
8	1 784	4/1	>30k	>30k	0.86	0	26.1
16	5 104	8/2	1 200	>30k	0.51	3	27.1
24	9 960	8/3	600	>30k	0.23	8	28.3
32	16 352	8/7	395	19 400 [10 000, 24 700]	0.22	8	29.5
48	33 744	8/7	250	8 900 [5 700, 23 200]	0.13	8	30.6
64	57 280	8/8	185	6 250 [5 350, 8 550]	0.05	8	32.5
96	122 784	8/8	130	6 125 [4 600, 8 100]	0.05	8	33.5
128	212 864	8/8	105	5 025 [3 300, 6 200]	0.06	8	33.5

$\log P$ is -0.46 (95% CI $[-0.66, -0.26]$), while for $d \geq 64$ it is -0.21 ($[-0.56, 0.04]$): the speed-up decelerates and cannot be distinguished from zero. t_{fit} keeps falling (-0.52 and -0.43), so the *relative* delay $t_{\text{gen}}/t_{\text{fit}}$ does not systematically grow: its slope is 0.08 ($[-0.10, 0.27]$) for $d \geq 32$ and 0.23 ($[-0.12, 0.47]$) for $d \geq 64$. Prediction S3 (relative delay increases with width) is therefore *not supported* at this resolution.

Norm at generalisation. The parameter norm at t_{fit} grows with width (25→48), as does the norm at initialisation, but the norm at t_{gen} varies only from 29.5 to 33.5 for $d \geq 32$ (coefficient of variation 5.6% over widths), whereas a critical *per-parameter* RMS would predict a $\sqrt{P}$-scaling—observed $\|\theta\|/\sqrt{P}$ falls 3× over the same range. We return to this in Sections 4.3 and 4.4.

4.2 The capacity gate across training-set sizes

We repeat the sweep at five further (p, frac) combinations covering $N = 432$–$2{,}823$ training pairs ($d \in \{8, \ldots, 48\}$, 3 seeds; runs stop 300 steps after t_{fit}, so t_{gen} is not measured here; the $(59, 0.4)$

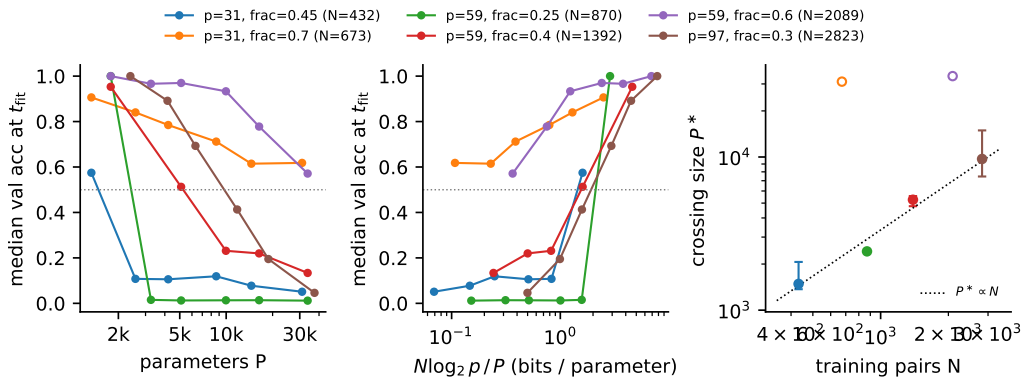


Figure 3: Memorise-first gate for different (p, frac) (3 seeds; $(59, 0.4)$: 8). (a) Median validation accuracy at t_{fit} vs P. (b) The same vs $N \log_2 p/P$: no collapse. (c) Size P^* at which (a) crosses 0.5 (log-linear interpolation, 95% bootstrap CI over seeds); open circles with arrows: no crossing in the tested range (lower bounds).

cell reuses E1). Runs that never fit count as "not memorise-first" ($a_{\text{fit}}^{\text{val}}$=1). H-CAP predicts that $a_{\text{fit}}^{\text{val}}$ falls through 0.5 at a critical size $P^* \propto N$, or equivalently that $a_{\text{fit}}^{\text{val}}$ collapses as a function of the stored-bits-per-parameter ratio $N \log_2 p/P$.

Memorise-first behaviour appears above a size threshold in every configuration where one is crossed (Fig. 3a), so the existence of a gate (C1) is robust. For the four configurations with $\leq$45% training data the crossings are $P^* = 1.5$k, 2.4k, 5.3k, 9.7k for $N = 432$, 870, 1,392, 2,823, a log-log slope of 1.04 (95% CI $[0.75, 1.29]$; 1000 bootstrap resamples over seeds; 4 points) and $N \log_2 p/P^* \approx 1.4$–2.1 bits per parameter, within the order of magnitude reported for language models (Morris et al., 2025) though we do not measure capacity directly. But the capacity picture is *incomplete*: the two configurations with larger training fractions (p=31, frac 0.7; p=59, frac 0.6) have no crossing up to P=3.1×10^4 and 3.4×10^4 although N lies between that of crossing configurations, and $a_{\text{fit}}^{\text{val}}$ does not collapse against $N \log_2 p/P$ (Fig. 3b). With more data the memorising solution itself generalises partially ($a_{\text{fit}}^{\text{val}} \geq 0.57$ at d=48 for frac 0.6), so size is not the only gate; the fraction of the table that is observed matters too, in line with the data-fraction dependence of grokking (Power et al., 2022; Liu et al., 2022). The slope is also threshold-dependent: 1.16 (4 configurations) for 0.3, 1.04 for 0.5 and 0.51 (5 configurations, high-fraction ones now included) for 0.7 (Appendix, `tables/e3_stats.json`). We therefore judge C2 *not supported as a general law*; it holds in the low-data subset only, with four points.

4.3 The optimiser clock

If generalisation requires a roughly fixed amount of weight-decay-driven cleanup, then once the model is wide enough that nothing else is limiting, t_{gen} should scale as $(\text{lr} \cdot \text{wd})^{-1}$. We vary lr$\in \{5\times10^{-4}, \ldots, 4\times10^{-3}\}$ and wd$\in \{0.5, \ldots, 4\}$ at $d \in \{32, 64, 128\}$ (3 seeds each; the baseline cell reuses E1 seeds 0–2; 7 further cells per width; Table 4 in the appendix).

At d=64 and 128, all runs generalise and $\log t_{\text{gen}}$ falls linearly in $\log(\text{lr} \cdot \text{wd})$ with slopes -1.24 ($[-1.42, -1.03]$) and -1.15 ($[-1.32, -0.95]$). Pooling both widths with a width fixed effect, the exponent is -1.14 ± 0.07 for lr and -1.25 ± 0.07 for wd (OLS standard errors, n=48): both near -1 but not exactly equal, and t_{gen} is not strictly a function of the product. At d=32 nine of 24 runs do not generalise in 30k steps, so no slope is estimated; the smaller model is slower at every setting (Fig. 4a). Across the 8$\times$ range of lr·wd the cell-mean norm at t_{gen} varies with a coefficient of variation of 6–8% within a width (max–min about 20%; Fig. 4c; 12% pooled over widths and cells, n=63), whereas t_{gen} changes by 13$\times$ at d=128.

Consequence for comparing sizes (S2). Hyper-parameters tuned per width erase most of the width effect among the larger models: the best median t_{gen} in the grid is 950 steps at d=64 and 850 at d=128 (3 seeds; the best cells sit at the edge of the grid, so the true optima may be lower for

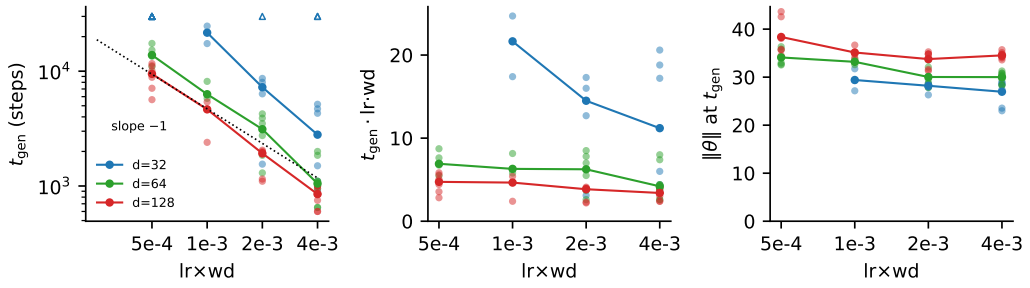


Figure 4: Optimiser clock (p=59, 3 seeds per cell). (a) t_{gen} vs lr·wd; open triangles: runs that did not generalise within 30k steps (all at d=32). Dotted: slope -1. (b) t_{gen}·lr·wd. (c) Parameter norm at t_{gen}.

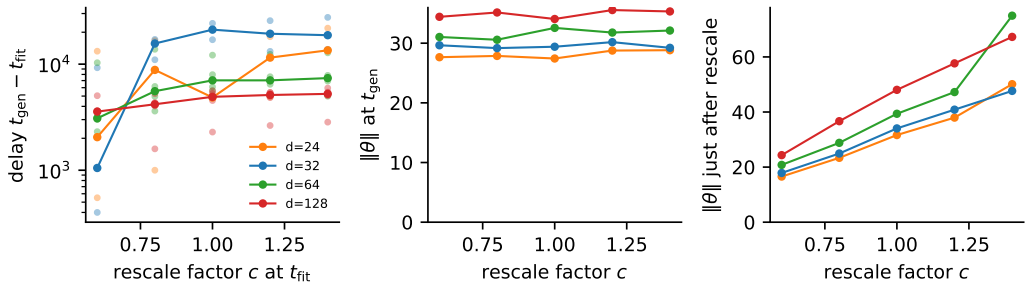


Figure 5: Rescaling intervention at t_{fit}. (a) Delay vs c (dots: runs, line: median; d=24, 32 include censored runs, shown as missing dots, and their medians use generalising runs only). (b) Norm at t_{gen}. (c) Norm immediately after rescaling.

both). At fixed lr/wd the same two widths differ by 1.2× (6,250 vs 5,025 in E1). The prediction that the speed-up survives tuning is thus only weakly supported for $d \geq 64$; at d=32 the best cell reaches 1,000 steps but only 2/3 seeds generalise there, so we cannot say whether width 32 is intrinsically slower once tuned.

4.4 Is norm a cause? A rescaling intervention

We rescale *all* parameters by $c \in \{0.6, 0.8, 1.2, 1.4\}$ at the first step where training accuracy reaches 0.99 (4 seeds; c=1 is E1 with the same seeds, so runs are paired up to the intervention).

Two results. First, the norm at t_{gen} is unaffected by c at every width (Fig. 5b), as H-NORM predicts. Second, the delay responds one-sidedly. Shrinking (c=0.6) shortens the delay by a factor $e^{0.67} \approx 2.0$ at d=64 (mean log-ratio -0.67, 95% CI $[-0.91, -0.35]$, n=4) and $e^{0.38} \approx 1.5$ at d=128 ($[-0.70, -0.03]$); c=0.8 gives -0.19 at d=128 ($[-0.32, -0.05]$) but a CI including 0 at d=64. Inflating (c=1.2, 1.4) changes the delay by $+5\%$/$+11\%$ at d=128 (CIs $[-0.01, 0.11]$ and $[0.04, 0.18]$ in log) and not detectably at d=64. A pure weight-decay-decay account would predict an increase of $\ln c/(\text{lr} \cdot \text{wd}) = 180$ and 336 steps, i.e. $+3.7\%$ and $+6.8\%$, close to what we see at d=128, whereas shrinking helps far more than $\ln c/(\text{lr} \cdot \text{wd})$ would give. **Caveat:** for $c \leq 0.8$ the rescaling is not a pure norm change—it knocks training accuracy down to 26–46% (c=0.6) and 81–94% (c=0.8), i.e. it partially destroys the memorising solution, which must be re-learnt. For $c \geq 1.2$ the fit is preserved (accuracy ≥ 0.91 after the step), but there the effect is small. So the experiment is consistent with the picture "norm decays until a width-independent threshold", but does not isolate norm from damage to the memoriser, and the one-sided response is what a decay-clock model would predict rather than evidence for a norm-triggered switch.

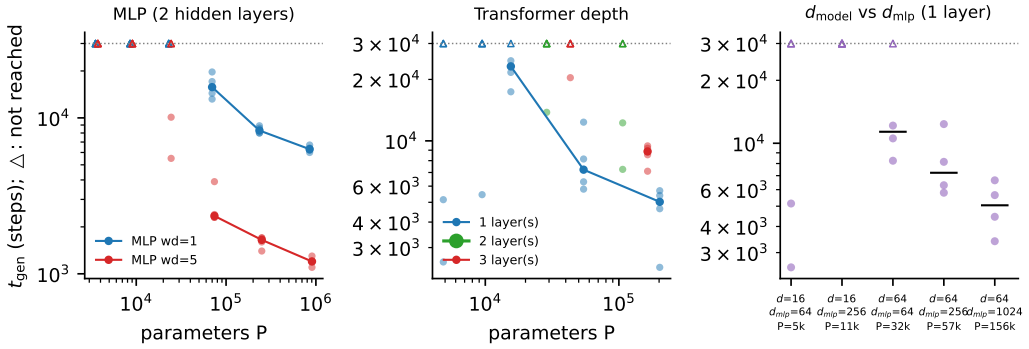


Figure 6: Other scale axes (4 seeds per point, p=59, frac 0.4, lr 10^{-3}). Dots: runs; triangles at the dotted line: did not generalise in 30k steps; line: median where finite. (a) 2-hidden-layer MLP at wd 1 and 5. (b) Transformer depth (1 layer: $d \in \{16, \ldots, 128\}$; 2 and 3 layers: $d \in \{32, 64\}$). (c) d_{mlp} varied at fixed d_{model}.

4.5 Other scale axes and architectures

MLP. A 2-hidden-layer ReLU MLP on concatenated embeddings has $a_{\text{fit}}^{\text{val}} \approx 0$ at every width ≥ 32 that fits at wd$=1$ (and ≥ 128 at wd$=5$), and at wd$=1$ none of the 12 runs with $d \leq 64$ generalises within 30k steps, while t_{gen} falls from 15,750 (d=128) to 8,300 (256) and 6,300 (512) (Fig. 6a). At wd$=5$ the order is the same (2,350, 1,650, 1,200), the time is 5.0–6.7$\times$ shorter than at wd$=1$ for a 5$\times$ larger wd, and d=64 now generalises in 2/4 runs. So the gate (small models fail to generalise), the fall of t_{gen} with size, its deceleration and the weight-decay clock all reappear in a different architecture; the medians give rough log-log slopes of -0.37 (wd 1) and -0.27 (wd 5) for $d \geq 128$ (3 widths, 4 seeds; no CI).

Depth. At fixed width, extra transformer layers do not help: at d=64 one layer generalises in 4/4 runs (median 7,225), two layers in 2/4 and three layers in 4/4 (median 8,875); at d=32, 3/4 (one layer) versus 1/4 (two and three layers) generalise within 30k steps, although P grows by 1.8–2.7$\times$. Total parameter count is therefore not the controlling variable (4 seeds each; the differences are suggestive, not significant).

d_{model} **versus** d_{mlp}**.** At d_{model}=64, raising d_{mlp} from 64 to 256 to 1024 gives $t_{\text{gen}} = 11{,}350$ (3/4 generalise), 7,225 (4/4), 5,050 (4/4), matching the d_{model} sweep at similar P (d=128: P=213k, 5,025). In the other direction, at d_{model}=16 a 16$\times$ MLP (11k parameters, 2.2$\times$ the baseline) is *not* better (0/4 generalise, $a_{\text{fit}}^{\text{val}} = 0.36$ vs 2/4 and 0.51), so a narrow residual stream limits generalisation even when the parameter count passes the gate value of the d_{model} sweep.

Other operations. Table 2 repeats E1 for $a - b$ and $a^2 + b^2$ (mod 59; 4 seeds, $d \in \{16, 32, 64, 128\}$; addition: E1 seeds 0–3). For subtraction every width that fits is memorise-first (including d=16, where addition has $a_{\text{fit}}^{\text{val}} = 0.51$), the time to generalise is longer than for addition, and generalisation appears only at larger widths (d=128: 3/4 runs, median of the generalising runs 17,750; d=64: 2/4; $d \leq 32$: 0/4)—the direction of S1 but a different, task-dependent gate. For $a^2 + b^2$, which has an eight-fold symmetry, no run is memorise-first ($a_{\text{fit}}^{\text{val}} \approx 0.7$–$0.8$ at every width, because the symmetry lets a memoriser answer many held-out pairs) and t_{gen} is long and non-monotone in width (e.g. 3/4 at d=16, 0/4 at d=32). So the gate is a property of the task and not only of N and P.

5 Scorecard

6 Discussion and limitations

What we can say. On modular addition with one-layer transformers trained by full-batch AdamW, "larger models grok" is true in two different senses. Below $\approx 10^4$ parameters (for N=1,392) there is little to grok: models cannot store the training labels, fit and generalisation coincide, and

Table 2: Other operations (p=59, frac 0.4, 4 seeds). fit/gen: runs reaching train≥0.99 / val≥0.99 within 30k steps; mem.: memorise-first runs; t: median over runs, censored runs counted as ∞; final val: median validation accuracy at the end.

op	d	fit/gen	mem.	t_{fit}	t_{gen}	$a_{\text{fit}}^{\text{val}}$	final val
$a+b$	16	4/2	1	1 425	>30k	0.51	0.80
$a+b$	32	4/3	4	450	23 175	0.17	0.99
$a+b$	64	4/4	4	190	7 225	0.03	1.00
$a+b$	128	4/4	4	105	5 025	0.05	0.99
$a-b$	16	4/0	4	3 025	>30k	0.08	0.16
$a-b$	32	4/0	4	450	>30k	0.02	0.10
$a-b$	64	4/2	4	200	>30k	0.01	0.91
$a-b$	128	4/3	4	110	17 750	0.00	0.99
a^2+b^2	16	4/3	0	750	24 550	0.81	0.99
a^2+b^2	32	4/0	0	330	>30k	0.72	0.96
a^2+b^2	64	4/3	0	170	19 375	0.70	0.99
a^2+b^2	128	4/4	0	100	19 650	0.69	0.99

Table 3: Pre-registered predictions and outcomes (Appendix A). "post-hoc" marks hypotheses written after inspecting E1 weight norms.

ID	Prediction	Outcome
C1	Memorise-first only above a critical size	Supported (Fig. 2c,d)
C2	$P^* \propto N$ across (p, frac)	Mixed: slope 1.04 [0.75,1.29] on 4 low-fraction configs, but two high-fraction configs have no crossing; threshold-sensitive
S1	t_{gen} falls with P (fixed lr, wd)	Supported for $d \in [32, 128]$; flat for $d \geq 64$ (CI includes 0)
S2	Speed-up survives per-width tuning	Weakly supported; ambiguous at grid edge
S3	$t_{\text{gen}}/t_{\text{fit}}$ increases with width	Not supported (slope CI includes 0)
M1	Norm at t_{gen} width-/optimiser-invariant (post-hoc)	Supported (CV 5.6% / 12%)
M2	Rescaling at t_{fit} shifts delay monotonically (post-hoc)	Partly: shrinking helps, inflating ≈no effect; confounded by damage to fit
M3	$t_{\text{gen}} \propto (\text{lr} \cdot \text{wd})^{-1}$	Supported by the pre-set criterion for $d \geq 64$ (slopes -1.24, -1.15 within $[-1.3, -0.7]$; the d=64 CI reaches -1.42)
GEN	Picture holds for MLP, other p/frac, depth	Partly: MLP yes (gate, saturation, clock); subtraction yes for speed but gate shifts; a^2+b^2 no memorise-first; depth no; gate not $\propto N$ across fractions

generalisation is often partial. Just above that size the delay is longest and runs often fail within our budget. Beyond it larger models generalise sooner, but the benefit saturates and, in our grid, nearly disappears once lr and wd are tuned: width acts through a prefactor on a clock set by $\text{lr} \cdot \text{wd}$, and the norm at which generalisation occurs is almost the same for all widths. Whether "scale makes grokking shorter" therefore depends on whether optimiser settings are held fixed, and a study that fixes them at one value can over-state the effect of width.

Limitations. (i) *Small compute, few seeds.* All results come from 4 CPU cores; 8 seeds in the main sweep and 3–4 elsewhere give wide intervals; several conclusions (S1 above d=64, S2, the sign of the rescaling effect at $c > 1$) are within noise. (ii) *Censoring.* A 30k-step budget censors runs near the gate; medians treat them as infinite, so the delay peak at $d \approx 24$–32 is a lower bound. (iii) *Single task and optimiser family.* Only modular addition is studied; training is full-batch AdamW with fixed β; other optimisers, mini-batch noise, parametrisations that preserve feature-learning rates across width (μP), and larger p are not studied. (iv) *Width is changed with fixed head count and* $d_{\text{mlp}} = 4d$; Section 4.5 varies the other axes only coarsely. (v) The capacity argument is a heuristic

and we do not measure bits per parameter; "capacity" enters only through P^*. (vi) H-NORM was formed after looking at weight norms in E1; its out-of-sample tests are E2 and E5, and the rescaling test is confounded for $c < 1$. (vii) The Fourier statistic is a proxy, not a full mechanistic analysis.

Reproducibility statement. All code, configurations, raw per-run curves and analysis scripts are included in the supplementary material; Appendix C lists dependencies, seeds, commands, run-times and the script generating each figure and table. Data are synthetic and produced by the code.

AI use statement. This research—idea formation, experimental design, code, execution of experiments, analysis and writing—was carried out end to end by an autonomous AI agent (Claude Sonnet 5.5, Anthropic) running unattended; no human edited the content. The agent wrote and ran all code and read all numbers in the paper from the raw run logs and the analysis outputs shipped with the code. References were checked with a literature-search tool (titles and abstract snippets only); bibliographic details beyond that were written from the agent's recollection and may contain errors. Generative AI was not used to produce any data. The submitting party is responsible for the final content.

Ethics statement. The work uses only synthetic data and small models and raises no ethical concerns that we are aware of.

REFERENCES

Xander Davies, Lauro Langosco, and David Krueger. Unifying grokking and double descent. *arXiv preprint arXiv:2303.06173*, 2023.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. In *International Conference on Learning Representations*, 2024.

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J. Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. In *Advances in Neural Information Processing Systems*, 2022.

Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations*, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

William Merrill, Nikolaos Tsilivis, and Aman Shukla. A tale of two circuits: Grokking as competition of sparse and dense subnetworks. *arXiv preprint arXiv:2303.11873*, 2023.

John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize? *arXiv preprint arXiv:2505.24832*, 2025.

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations*, 2020.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv preprint arXiv:2206.04817*, 2022.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems*, 2022.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.

Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. In *Advances in Neural Information Processing Systems*, 2023.

A Pre-registration and deviations

The hypotheses in the Introduction were written to `PREREG.md` while the main width sweep (E1) was running, before any E1 result was inspected, and are reproduced in the supplementary material. They were formulated *after* a one-seed-per-width pilot (p=59, same settings, `results/pilot2`) which suggested that larger widths fit early and generalise sooner and that the smallest widths fit and generalise jointly; the direction of S1 is therefore not a blind prediction, but it is tested on 8 fresh seeds and on other settings. An addendum to `PREREG.md`, written after E1 was nearly complete and its weight norms had been inspected, adds M1–M3 (E2 and E5 were then designed to test them out-of-sample). Deviations and failures: (1) the planned d=12 and d=192 points of E1 were dropped when we discovered the container is limited to 4 CPUs (not the 40 reported by `nproc`); (2) the first launch of E3 crashed because d=6 is not divisible by 4 heads, we removed d=6 and re-ran E3 (completed runs were reused); (3) the one-seed pilot also included an MLP sweep at wd$= 1$ in which widths ≤ 64 had not generalised after 20k steps—this is why the MLP experiment in Section 4.5 also uses wd$= 5$.

B Additional tables

C Reproduction details

The supplementary material contains the full repository (`README.md`): environment (Python 3.11.2, PyTorch 2.15.0 CPU, numpy 2.4.6, scipy 1.17.1, matplotlib 3.11.2, pandas 3.1.0), the exact commands, seeds, estimated cost and a figure/table-to-script map. Data are generated in memory (no download, no version). Runs take 1–40 minutes on one CPU thread; the whole study used about 47 CPU-hours (Table 6). Raw per-run learning curves for every experiment are included (`results/*/*.json`), so all analysis can be re-run without retraining. Seeds: model initialisation `torch.manual_seed(seed)`, split `Generator(10000+seed)`. The one-seed exploratory pilot (`results/pilot2`) used an earlier version of the logging code and is not used for any reported number except in the narrative of Appendix A.

Table 4: Optimiser clock (E2), medians over 3 seeds per cell. "–": no run generalised. clock $= t_{\text{gen}} \cdot \text{lr} \cdot \text{wd}$; $\|\theta\|_{\text{gen}}$: mean norm at t_{gen} over generalising runs.

d	lr	wd	lr·wd	gen	t_{fit}	t_{gen}	clock	$\|\theta\|_{\text{gen}}$
32	5e-04	1	5.0e-04	0/3	800	–	–	–
32	1e-03	0.5	5.0e-04	0/3	380	–	–	–
32	1e-03	1	1.0e-03	3/3	450	21 650	21.7	29.4
32	1e-03	2	2.0e-03	2/3	550	4 400	8.8	29.0
32	1e-03	4	4.0e-03	2/3	650	1 000	4.0	23.3
32	2e-03	1	2.0e-03	3/3	230	8 000	16.0	27.7
32	2e-03	2	4.0e-03	2/3	330	3 325	13.3	28.0
32	4e-03	1	4.0e-03	3/3	150	4 300	17.2	28.8
64	5e-04	1	5.0e-04	3/3	340	13 750	6.9	32.7
64	1e-03	0.5	5.0e-04	3/3	170	15 250	7.6	35.5
64	1e-03	1	1.0e-03	3/3	190	6 300	6.3	33.2
64	1e-03	2	2.0e-03	3/3	210	2 750	5.5	30.1
64	1e-03	4	4.0e-03	3/3	290	1 000	4.0	29.2
64	2e-03	1	2.0e-03	3/3	110	3 500	7.0	30.0
64	2e-03	2	4.0e-03	3/3	130	950	3.8	30.3
64	4e-03	1	4.0e-03	3/3	80	1 100	4.4	30.5
128	5e-04	1	5.0e-04	3/3	180	11 150	5.6	36.5
128	1e-03	0.5	5.0e-04	3/3	100	8 850	4.4	40.2
128	1e-03	1	1.0e-03	3/3	110	4 650	4.7	35.1
128	1e-03	2	2.0e-03	3/3	110	1 900	3.8	32.7
128	1e-03	4	4.0e-03	3/3	130	850	3.4	34.4
128	2e-03	1	2.0e-03	3/3	70	2 000	4.0	34.8
128	2e-03	2	4.0e-03	3/3	70	850	3.4	34.5
128	4e-03	1	4.0e-03	3/3	50	1 000	4.0	34.8

Table 5: Rescaling intervention (E5), 4 seeds per cell, c=1 from E1 (same seeds). Delay $= t_{\text{gen}} - t_{\text{fit}}$ (median over generalising runs).

d	c	gen	t_{fit}	t_{gen}	delay	$\|\theta\|_{\text{gen}}$
24	0.6	3/4	925	2 900	2 050	27.6
24	0.8	2/4	850	9 650	8 825	27.9
24	1.0	1/4	600	5 450	4 850	27.4
24	1.2	2/4	600	12 125	11 525	28.8
24	1.4	2/4	650	14 150	13 500	28.8
32	0.6	3/4	675	1 700	1 050	29.6
32	0.8	3/4	600	16 300	15 650	29.2
32	1.0	3/4	450	21 650	21 150	29.4
32	1.2	3/4	450	19 850	19 350	30.2
32	1.4	3/4	450	19 250	18 750	29.2
64	0.6	4/4	290	3 375	3 085	31.0
64	0.8	4/4	240	5 800	5 560	30.6
64	1.0	4/4	190	7 225	7 035	32.6
64	1.2	4/4	190	7 225	7 035	31.8
64	1.4	4/4	210	7 600	7 385	32.1
128	0.6	4/4	160	3 725	3 570	34.4
128	0.8	4/4	115	4 300	4 185	35.1
128	1.0	4/4	105	5 025	4 920	34.1
128	1.2	4/4	105	5 225	5 120	35.6
128	1.4	4/4	105	5 350	5 245	35.3

Table 6: Compute per experiment (sum of per-run wall time on a 4-core quota container, which includes CPU contention).

exp.	content	runs	core-hours
E1	width sweep, p=59	64	8.3
E2	lr/wd clock grid	63 (+9 copied from E1)	6.2
E3	gate vs N	90	2.8
E4	MLP, depth, d_{mlp}	76	10.7
E5	rescaling	64	6.9
E6	other operations	32	10.5